9/30/2026

Mini Project 2:
Pennine Pizza Works

Seerat, Aiman, Jimmy, Faris

Content Table:

Analysis:.....2
Requirements:.....2
IPO Table:.....3
Design:.....4
Function/subroutine list:.....4
Data Plan.....5
Planned main Program flow:.....6
Build:.....7
Screenshots of program running:.....15
Testing.....18
Introduction.....18
Test table.....19
Testing evidence.....20
Deploy:.....32
Individual Evidence:.....32
Individual Reflection - Jimmy:.....33
Individual Reflection - Faris:.....33
Individual Reflection - Seerat:.....33
Individual Reflection - Aiman:.....33

Introduction:

This document details the System Analysis and Requirements Specification for a new point of sale order management system designed for Pennine Pizza Works. The primary purpose of this software is to transition the business away from risky manual calculations by establishing a semi-automated, computerised ordering process. By

defining explicit data requirements upfront, this document ensures the final application successfully captures precise customer profiles, handles complex ordering logic, and mitigates the threat of staff calculation errors.

The scope of the system focuses entirely on streamlining order accuracy while strictly enforcing the client's commercial business rules. The application handles order entry validates constraints such as limiting transactions to between 1 and 6 pizzas, and computes pricing dynamically based on pizza size and extra toppings. Additionally, the software acts as an automated rule engine programmatically applying a 10% discount for orders over £20 and instantly injecting a £2.50 delivery fee when required as well as culminating in an accurate, itemised bill total.

Analysis:

Requirements:

The requirements were obtained from the Pennine Pizza Works client brief. This system must be able to gather customer and order details, execute price calculations based on the business rules and produce an accurate and detailed bill. The input, processing and output requirements were determined to make sure that the system fulfils the client's needs as well as to minimize the risk of calculation error in the order processing when orders are manually input.

No.	Requirement	Type
1	The program must allow staff to enter the customer's name.	Input
2	The program must allow staff to enter the customer's address.	Input
3	The program must allow staff to enter the customer's phone number.	Input
4	The program must allow staff to enter the number of pizzas ordered.	Input
5	The program must allow staff to enter the size of each pizza (Small, Medium, or Large).	Input
6	The program must allow staff to enter the number of extra toppings for each pizza.	Input
7	The program must allow staff to select whether delivery is required.	Input
8	The program must validate that the number of pizzas entered is between 1 and 6.	Process
9	The program must calculate the standard cost of each pizza based on its size.	Process
10	The program must calculate the cost of extra toppings for each pizza.	Process
11	The program must calculate the total cost of all pizzas and toppings.	Process
12	The program must calculate the order subtotal before any discount or delivery charge is applied.	Process
13	The program must apply a 10% discount if the order subtotal exceeds £20.	Process
14	The program must calculate and add a £2.50 delivery charge when delivery is requested.	Process
15	The program must calculate the final total cost of the order.	Process
16	The program must display the customer's name, address and phone number on the bill.	Output
17	The program must display the cost of each pizza ordered.	Output

18	The program must display the cost of extra toppings.	Output
19	The program must display the order subtotal before discount and delivery charges.	Output
20	The program must display any discount applied to the order.	Output
21	The program must display any delivery charge applied to the order.	Output
22	The program must display the final total cost of the order.	Output
23	The program must produce a personalised and itemised bill for the customer.	Output
24	The program must validate all numeric inputs and request re-entry when invalid data is entered.	Process

IPO Table:

Input	**Process**	**Output**
Customer Name	Store the name	Customer name displayed on order
Customer Address	Store the address	Delivery address displayed
Customer Phone Number	Store the number	Contact number displayed
Pizza Quantity (1-6)	Validate: Minimum =1 Maximum =6	Number of pizzas
Pizza size (Small, medium, large)	Small = £3.25, Medium = £5.50 Large = £7.15	Adjusted pizza price
Number of extra toppings	0 toppings = £0.00 1 toping =£ 0.75 2 toppings = £1.35 3 toppings = £2.00 4 or more = £2.50	Extra toppings cost
Delivery request	If yes £2.50 charge. If no= £ 0.00	Delivery confirmed
Discount	If subtotal is over £20.00, then =10% of subtotal. If not, d £0.00	Discount amount applied
Pizza size and extra toppings	Pizza cost = size price + topping price	Cost of each pizza

All pizza costs	Subtotal = add up all pizza cost	Subtotal displayed
Subtotal, discount and delivery charge	final total =subtotal – discount+ delivery charge	Final order total displayed

Design:

Function/subroutine list:

Name	Takes in	Gives back	Function / Subroutine	Requirements covered
GetCustomerDetails (customer)	The customer arrays	Nothing it fills the array	Subroutine	R1, R2, R3
GetValidNumber (min, max)	Min, max (e.g. 1-6 for pizzas, 0-10 for toppings)	The valid Number	Function	R4, R6, R8, R24
GetSizePrice (size)	The size (Small/Medium/Large)	It gives back the price (£3.25/ £5.50 / £7.15)	Function	R5, R9
GetToppingPrice (numToppings)	Number of toppings (0-10)	The topping Price (£0 /£0.75 / £1.35 / £2.00 / £2.50)	Function	R10
GetDeliveryCharge ()	Nothing	£0.00 or £2.50	Function	R7, R14
CalcCosts (sizeCost, toppingCost, numPizzas, deliveryCharge, costs)	Price of each pizza, the number of pizza and delivery charge	Nothing , fills the costs array	Subroutine	R11, R12, R13, R14, R15
DisplayBill (customer, sizeCost, toppingCost, numPizzas , costs)	Customer array, size and topping costs arrays, number of pizzas, cost array	Nothing Print the itemised bill	Subroutine	R16, R17, R18, R19, R20, R21, R22, R23

- **GetCustomerDetails (customer):** Asks for the customer's name, address and phone number and stores them in the customer array.

- **GetValidNumber** (min, max): Asks the user to enter a number and checks it is between the minimum and maximum. If the number is outside the limits, it keeps asking until a valid number is entered.

- **GetSizePrice: (size):** Receives the Pizza size (Small, Medium or Large) and returns the price for that size (£3.25, £5.50 or £7.15).

- **GetToppingPrice (numToppings)**: Receives the number of extra toppings (0-10) and returns the total topping price for that pizza (£0.00, £0.75, £1.35, £2.00 or £2.50 for 4 more).

- **GetDeliveryCharge():** Asks if the customer wants delivery (yes/no) and returns £2.50 if yes or £0.00 if no.

- **CalcCosts(sizeCost, toppingCost, numPizzas, deliveryCharge, costs):** Adds up the size and the topping price of every pizza to get the Subtotal. if the subtotal is over £20 it applies a 10% discount, otherwise the discount is £0.00. It then adds the delivery charge and calculates the final total (subtotal - discount + delivery). All results are stored in the costs array.

- **DisplayBill (customers ,SizeCost , toppingCost,NumPizzas,costs)**: Prints the personalized, itemised bill. It reads the customer array and the costs array. It uses the size cost and topping cost of each pizza and the number of pizzas. All money is shown to 2 decimal places. From the costs array it prints the Subtotal, discount, delivery charge and final total. All money is shown to 2 decimals places.

Data Plan

Array name	Data type	Size	Index holds
customer	String	3	[0] Name, [1] address, [2] phone number
SizeCost	Real	6	Size price of each pizza ([0] =pizza 1….[5] =pizza 6)
toppingCost	Real	6	Topping price of each pizza ([0] = pizza 1 [5] = pizza 6)

costs	Real	4	[0] subtotal, [1] discount, [2] delivery charge, [3] final total

Variable	Holds	Type
numPizzas	How many pizzas (1-6)	Integer
size	Small / Medium / Large	String
numToppings	Number of toppings (0-10)	Integer
deliveryCharge	£2.50 or £0.00	Real

Array were used because a customer can order up to 6 pizzas. Without the two arrays, we would need 12 separate variables for the size costs and topping costs. With arrays, a FOR loop can be used to store each price; without them, we would have to write the same code 6 times.

Planned main Program flow:

Start
Declare Variables
Get Customer Name
Get Customer Address
Get Customer Phone Number
Input Number of Pizzas
Validate Pizza Quantity (1-6)
FOR Each Pizza
  Input Pizza Size
  Validate Pizza Size (S, M or L)
  Calculate Pizza Size Price
  Input Number of Extra Toppings
  Calculate Topping Price
  Calculate Pizza Line Total
  Add Line Total to Order Subtotal
NEXT
Input Delivery Choice
Calculate Delivery Charge
Check If Subtotal Is Greater Then £20

```
If True
   Apply 10% Discount
Else
   Set Discount To £0
End If
Calculate Final Total
Display Itemised Bill
End
```

The program starts by asking for the customer information and confirming the quantity of pizza ordered. Each pizza is then processed in turn using a loop, performing the following checks: Pizza size checking, Topping selection and calculation of cost. After all pizzas have been processed, the program works out any discounts and delivery charge and then sends out a personalised broken-down receipt showing the total order.

Build:

The Pizza ordering application was constructed within Flowgorithm based on the verified requirements.

The Figures below illustrate the complete visual source layout of the system, showcasing the declaration blocks, loops and conditional logic.

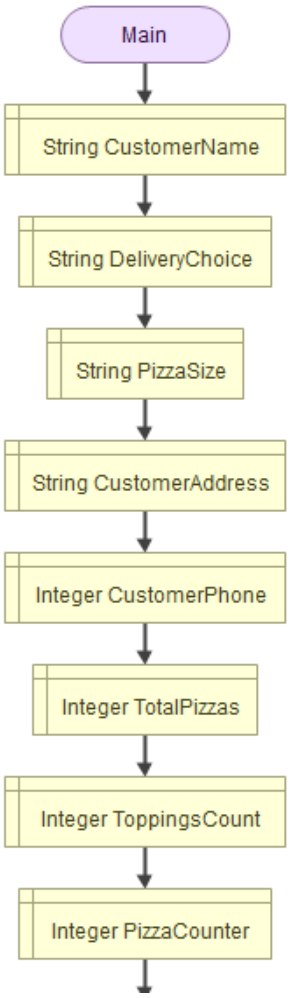


Figure 1a: *Main variable declaration - string and integer setups* (Source: Author, 2026)

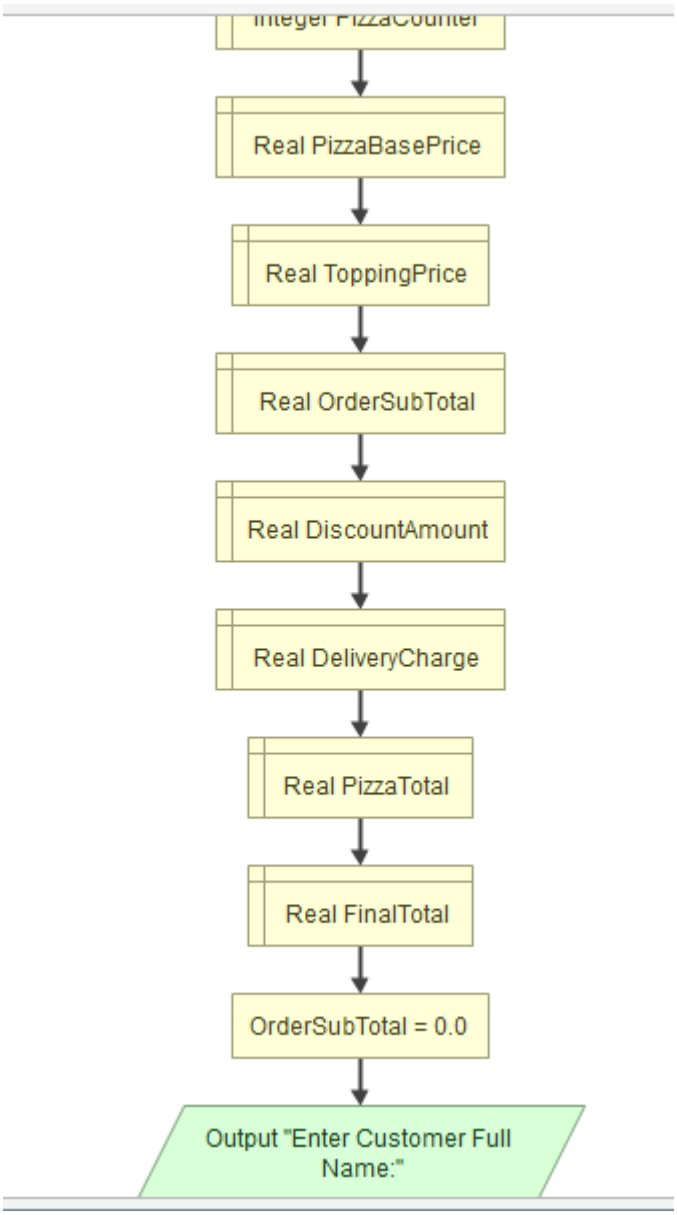


Figure 1b: *Main Variable declaration – Real price setups and initial data values* (Source: Author, 2026)

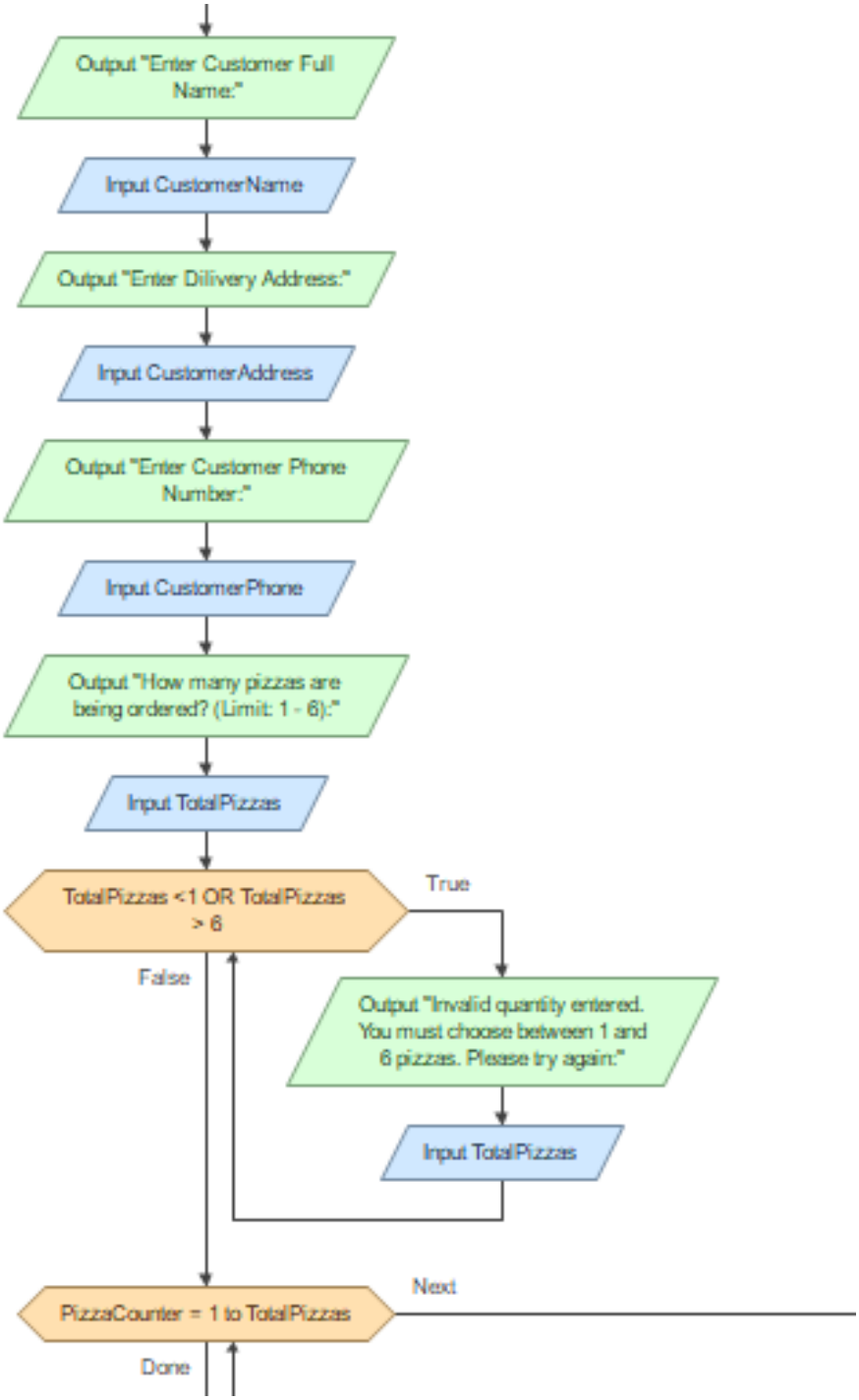


Figure 2: *Customer info capture and Pizza quantity check loop* (Source: Author, 2026)

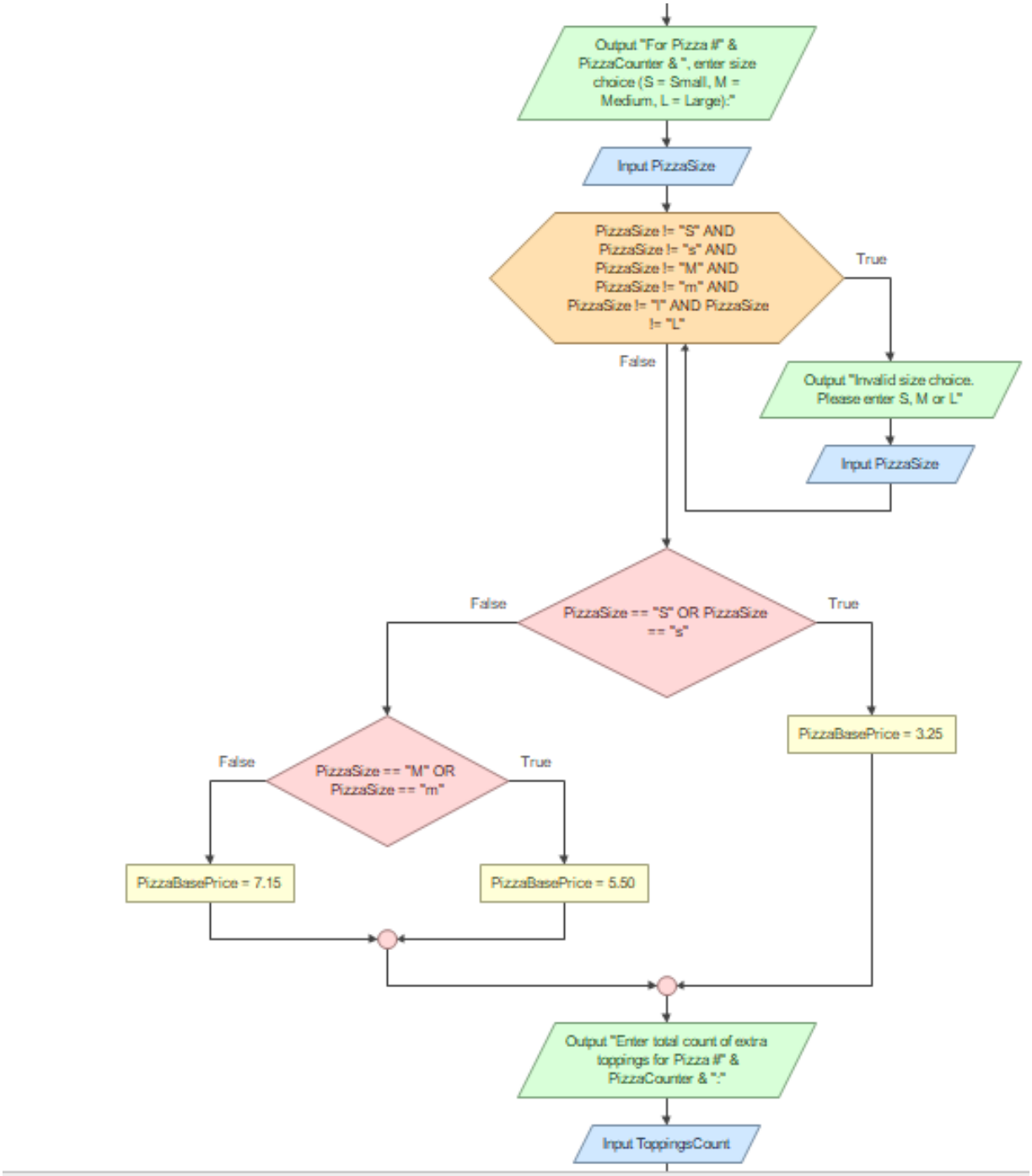


Figure 3: *Main loop for choosing pizza size and checking for valid sizes* (Source: Author, 2026).

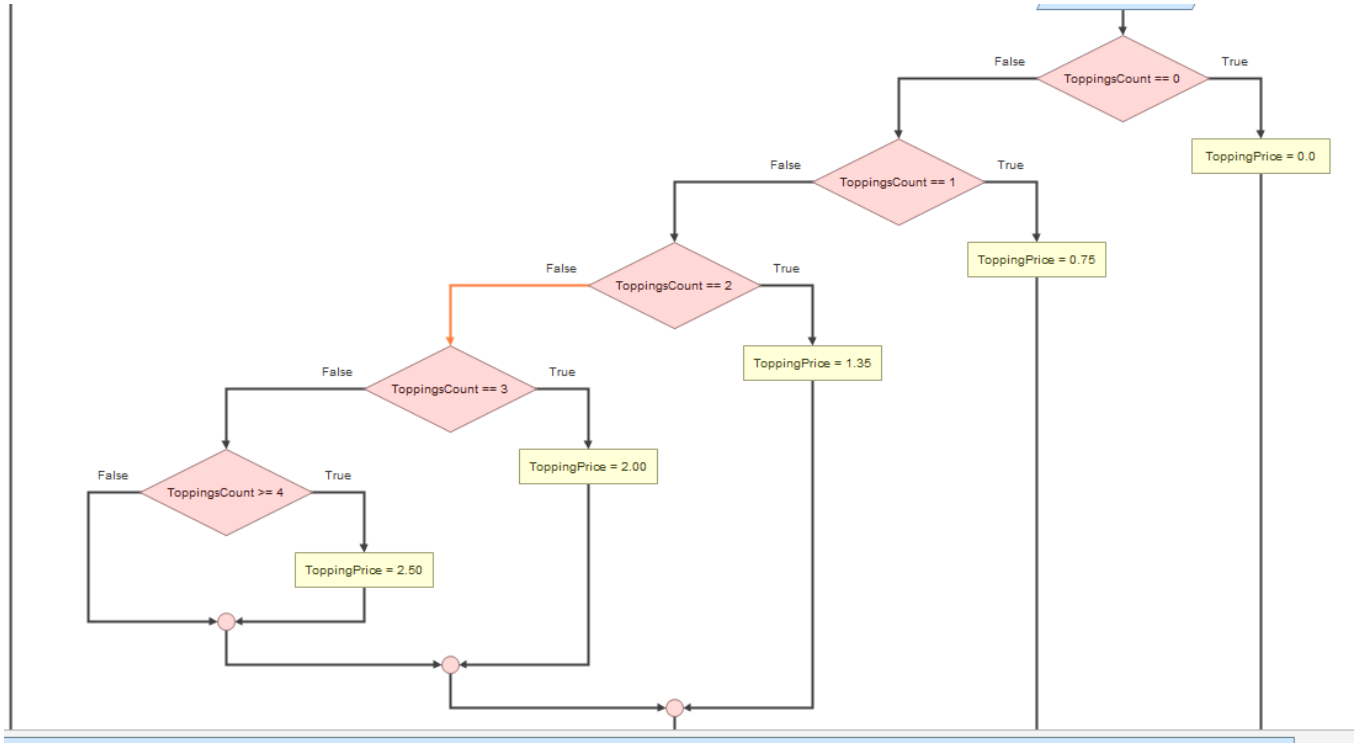


Figure 4: *The topping counter check loop and pricing levels* (Source: Author, 2026).

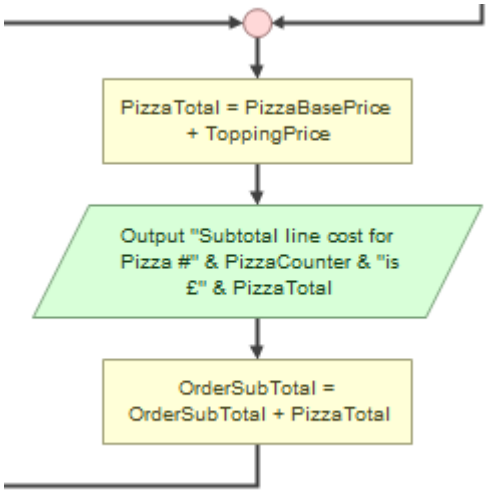


Figure 5: *Adding up individual pizza totals and updating the order subtotal* (Source: Author, 2026).

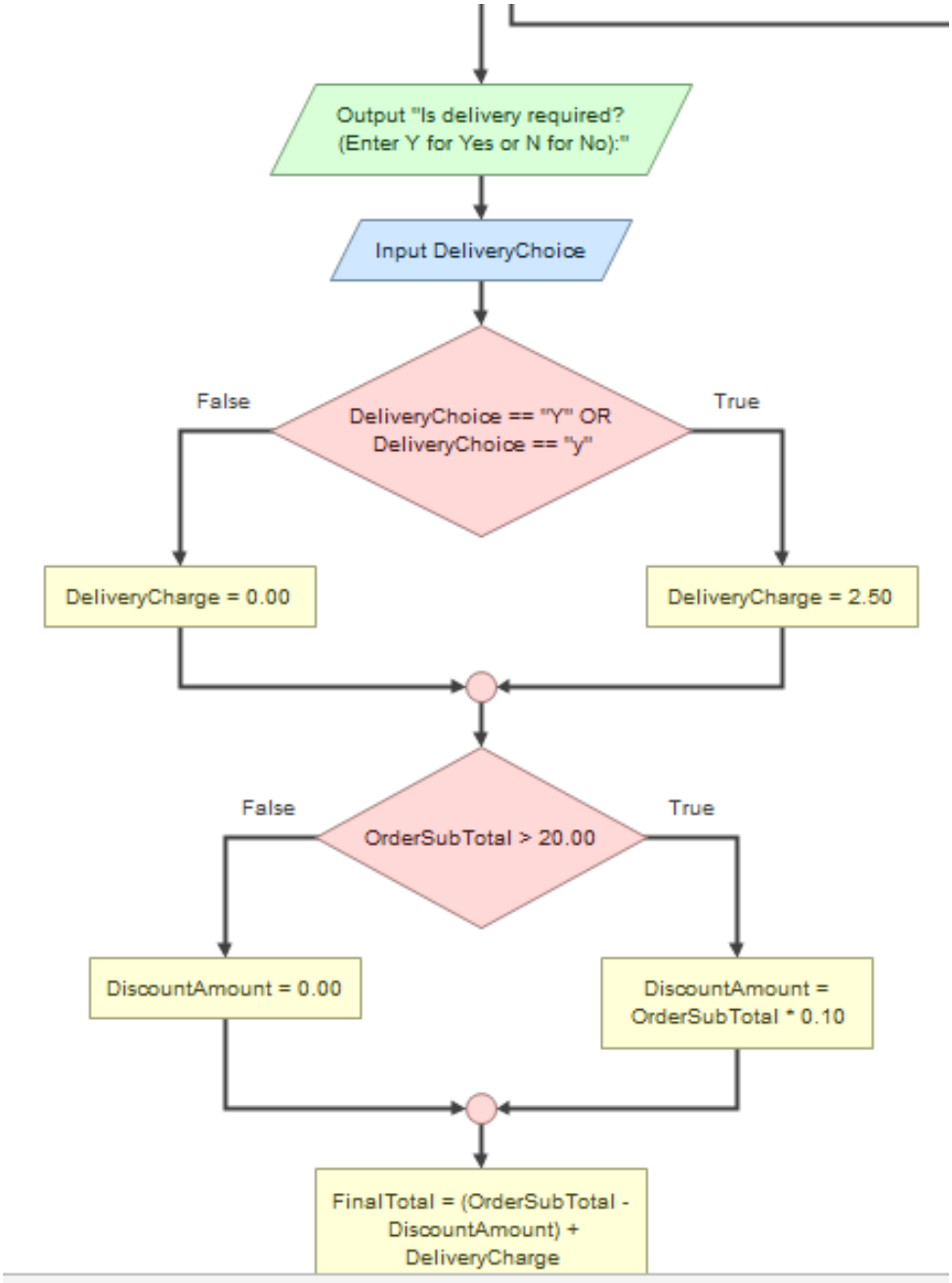


Figure 6: *Delivery choice check and the 10% promotional discount check* (Source: Author, 2026).

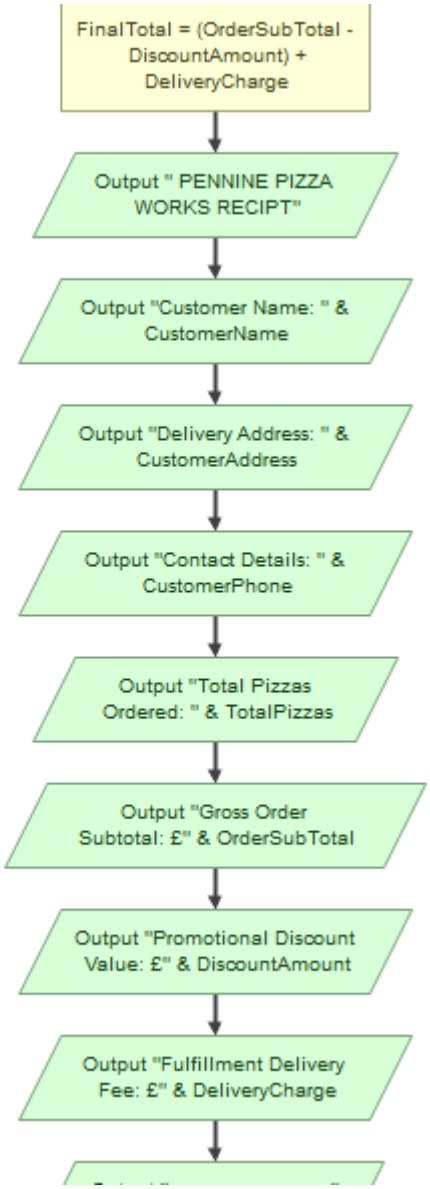


Figure 7a: *Final order total calculation and printed receipt layout* (Source: Author, 2026).

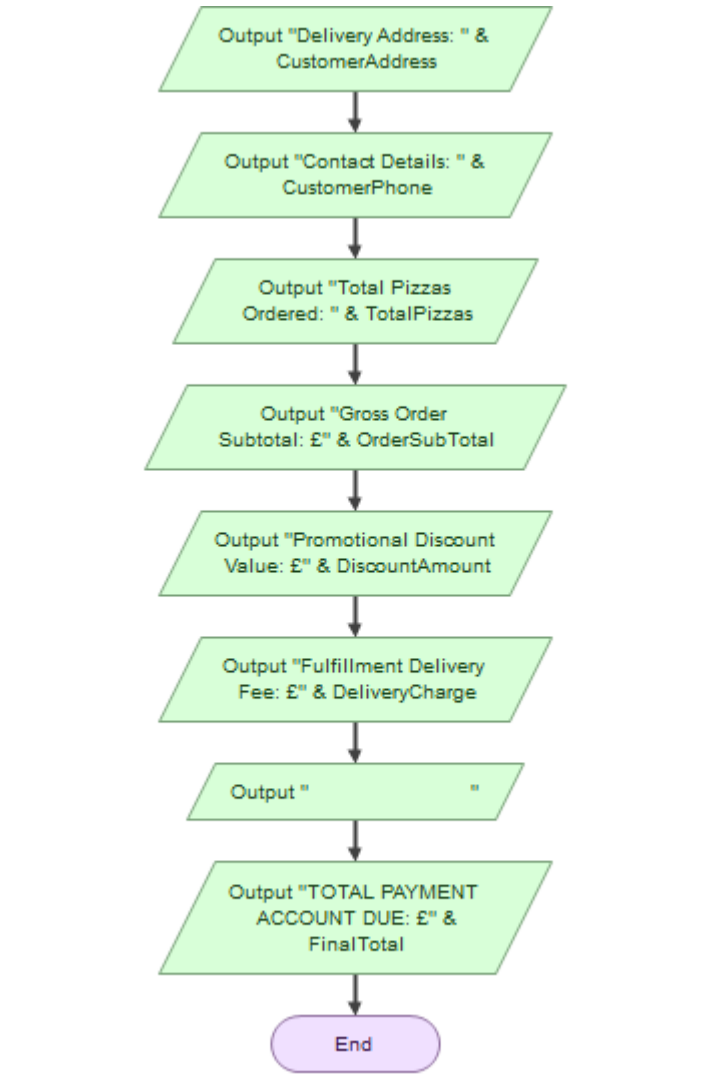


Figure 7b: *Final receipt layout* (Source: Author, 2026).

Screenshots of program running:

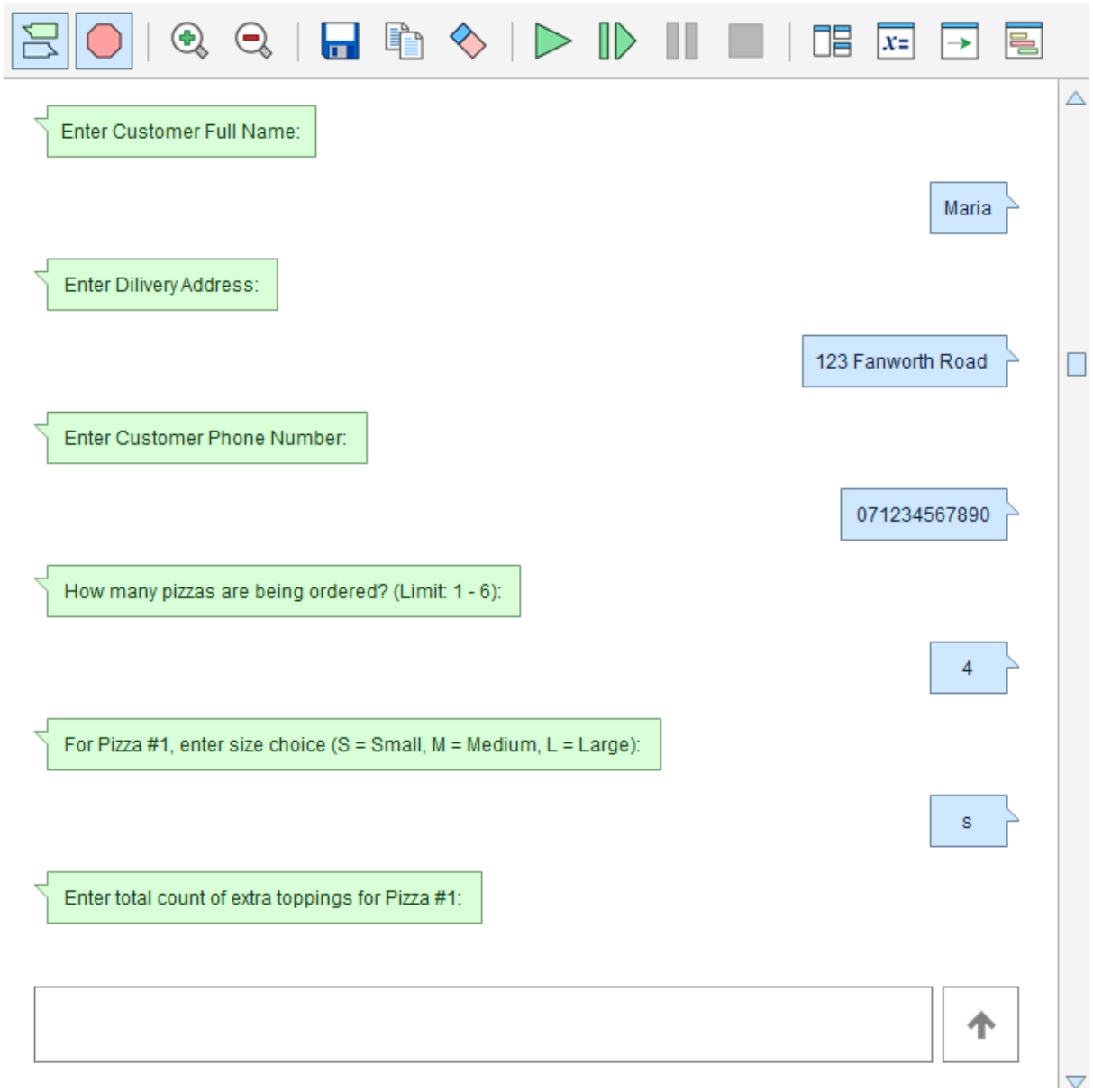


Figure 8: *Entering customer details and validating an order quantity of 4 pizzas* (Source: Author, 2026).

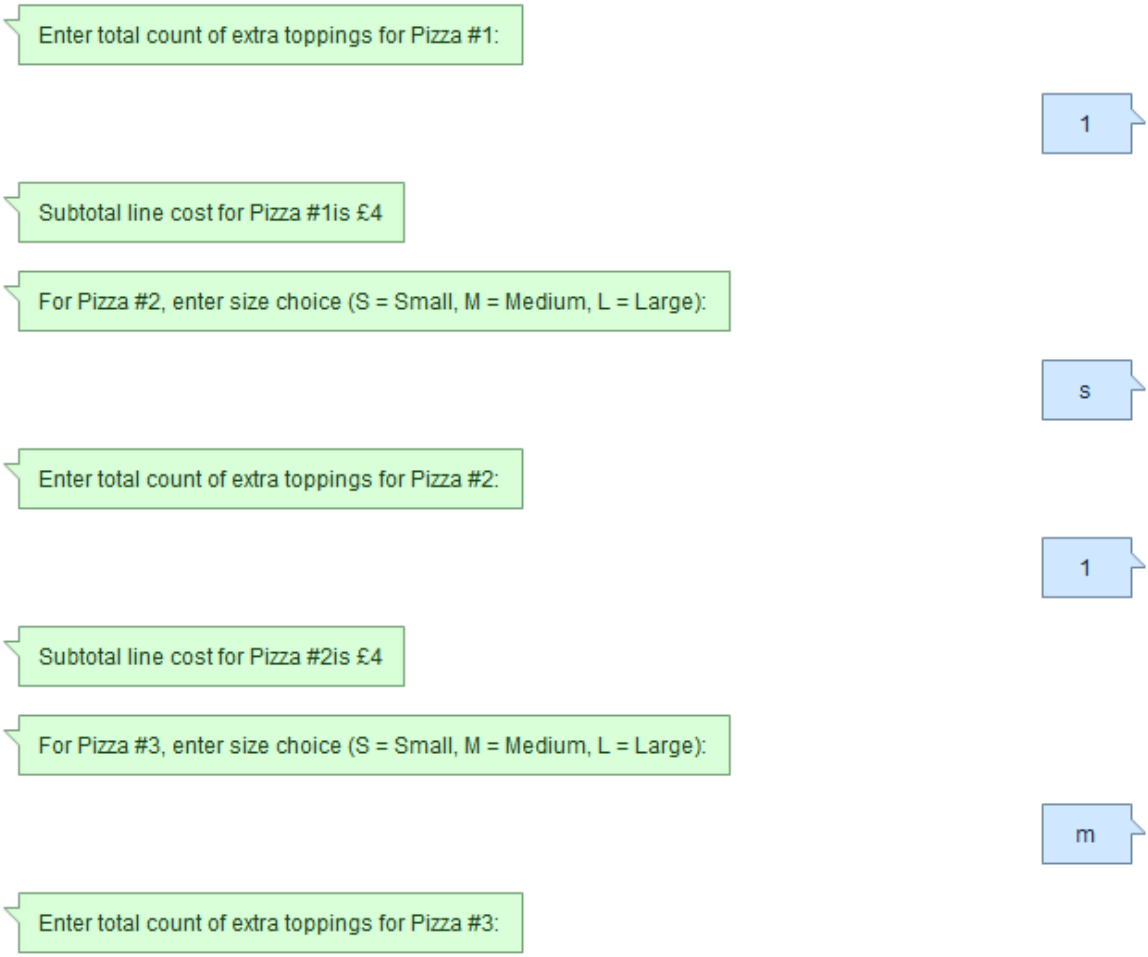


Figure 9: *Entering pizza sizes and displaying the line costs for pizza 1 and pizza 2* (Source: Author, 2026).

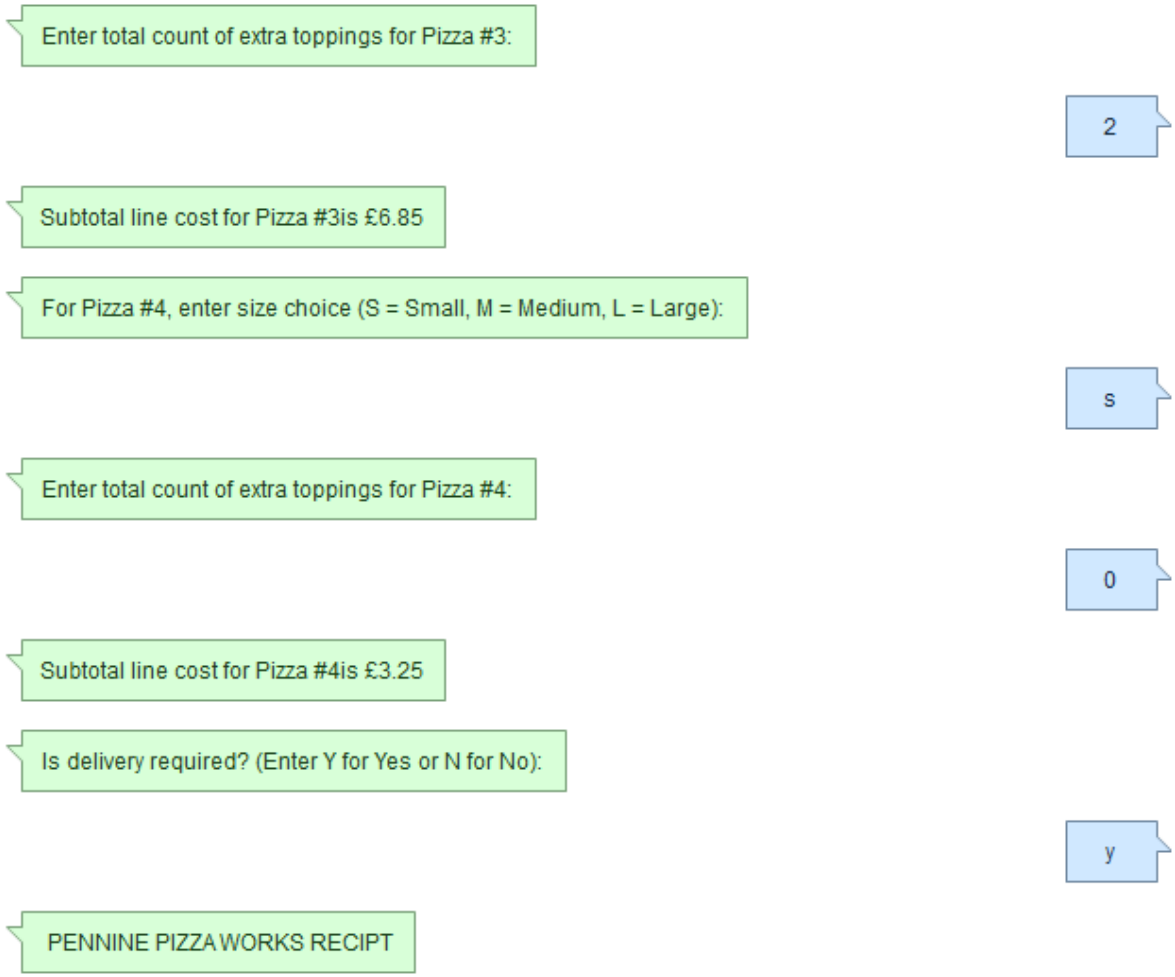


Figure 10: *Processing pizza 3 and pizza 4-line costs followed by the delivery prompt* (Source: Author, 2026).

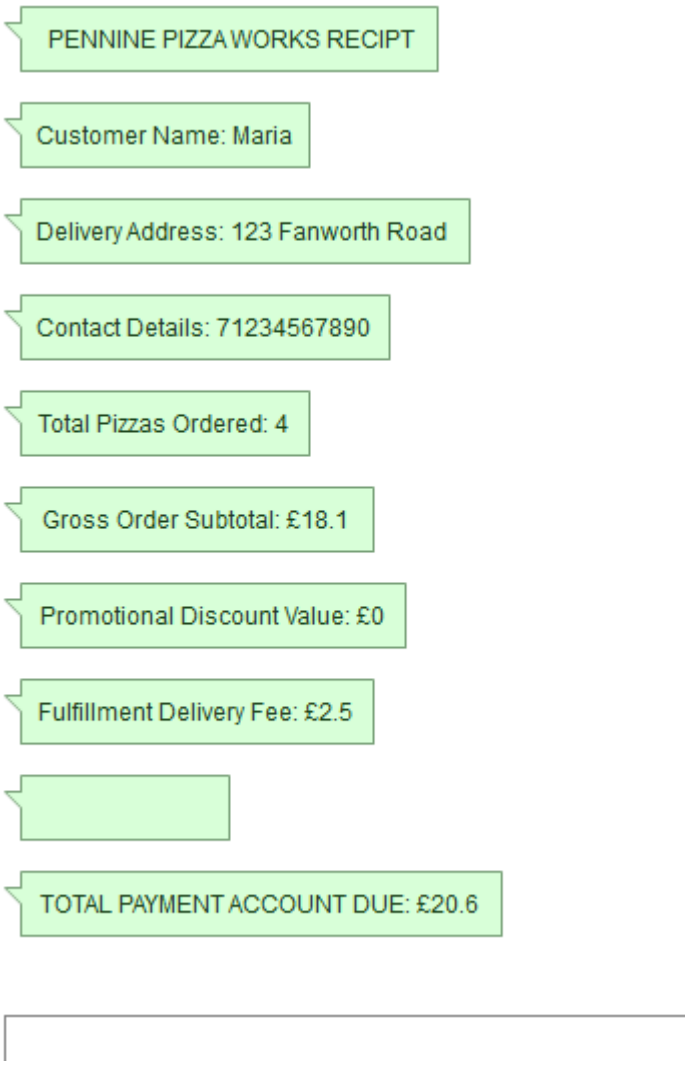


Figure 11: *The final printed customer receipt showing accurate calculations and decimal outputs* (Source: Author, 2026).

Testing

Introduction

The whole purpose of testing is to check whether the Pennine pizza works ordering system meets the requirements of the client and calculates the customers' orders correctly. A test plan has been prepared using normal extreme and abnormal data to check pizza sizes quantities topping charges delivery costs and discounts.

Individual contribution

My allocated role was testing I am responsible for preparing test cases to check whether the program meets the client's requirements. I have also prepared a structured test plan to help identify possible errors that could be in the system.

Test table

Test no.	Test type	Test input	Expected result	Actual result	Pass/fail
1	normal	Pizza quantity = 2	Program excepts 2 pizzas	Program accepted 2 pizzas and moved to delivery	Pass
2	Abnormal	Pizza quantity = -1	Program rejects input and asks again	Program rejected 1 and asked for a valid quantity	Pass
3	Extreme	Pizza quantity = 1	Program accepts 1 pizza	Program accepted 1 pizza	Pass
4	Extreme	Pizza quantity= 6	Program accepts 6 pizzas	Program accepted 6 pizzas	Pass
5	Abnormal	Pizza quantity =7	Program rejects input and asks again	Program rejected 7 and requested a valid quantity	Pass
6	Normal	Pizza size = small	Pizza base price is £3.25	The program displayed £3.25	pass
7	Normal	Pizza size = medium	Pizza base price is £5.50	Program displayed £5.50	Pass
8	Normal	Pizza size = large	Pizza price is £7.15	Program displayed £7.15	Pass
9	normal	Extra toppings = 2	Topping charge is £1.35 per pizza	Program displayed £4.60	Pass
10	Normal	Extra toppings = 0	Topping charge is £0.00	Program accepted 0 toppings without adding a topping charge	Pass
11	Normal	Delivery = yes	£2.50 delivery charge added	Program correctly added a 2.50 delivery charge	Pass

12	Normal	Delivery = no	No delivery charge added	Program correctly applied no delivery charge	Pass
13	Normal	Subtotal = £25.00	£2.50 discount applied	Program applied a £2.74 discount to a £27.40 subtotal	Pass
14	Extreme	Subtotal = £ 20.00	No discount applied	No discount applied	Pass
15	Abnormal	Pizza size = xl	Invalid size rejected and user asked again	Program rejected xl and asked the user to enter a valid pizza size (s,m or l)	Pass

Testing evidence

When 2 pizzas was selected the program correctly requested details for delivery.

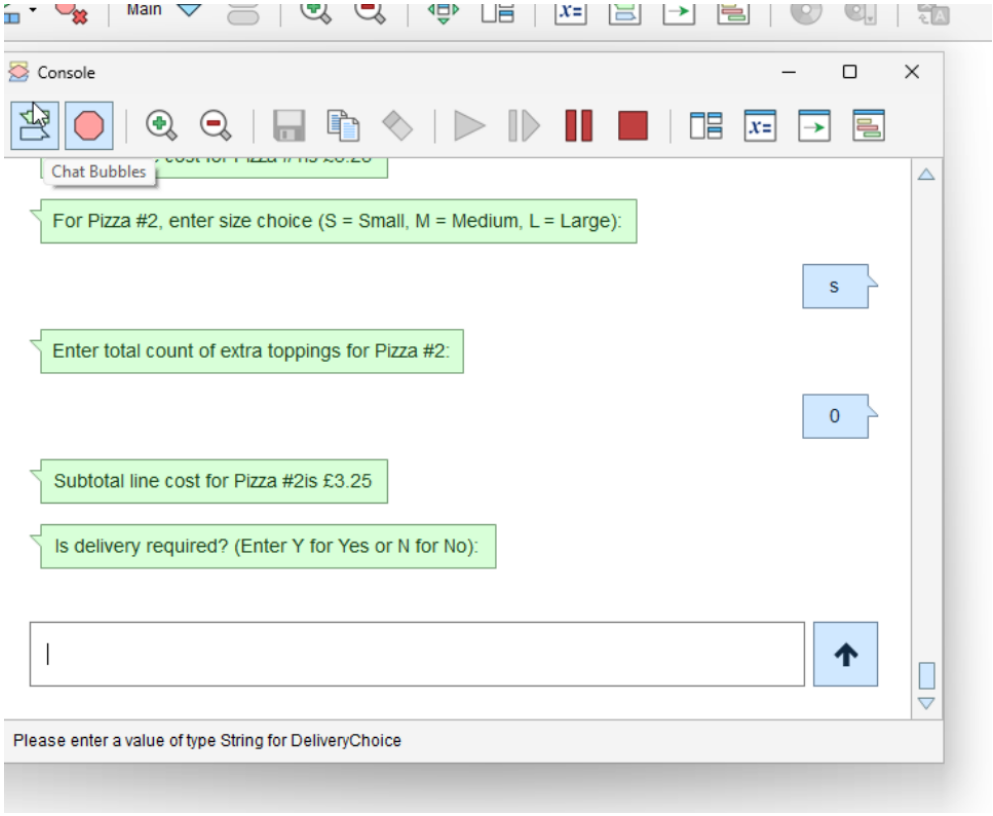


Figure 2

The program was tested using an invalid pizza quantity of –1. The program correctly rejected and it asked the user to enter a valid quantity between 1 – 6. this confirms that the input validation works correctly.

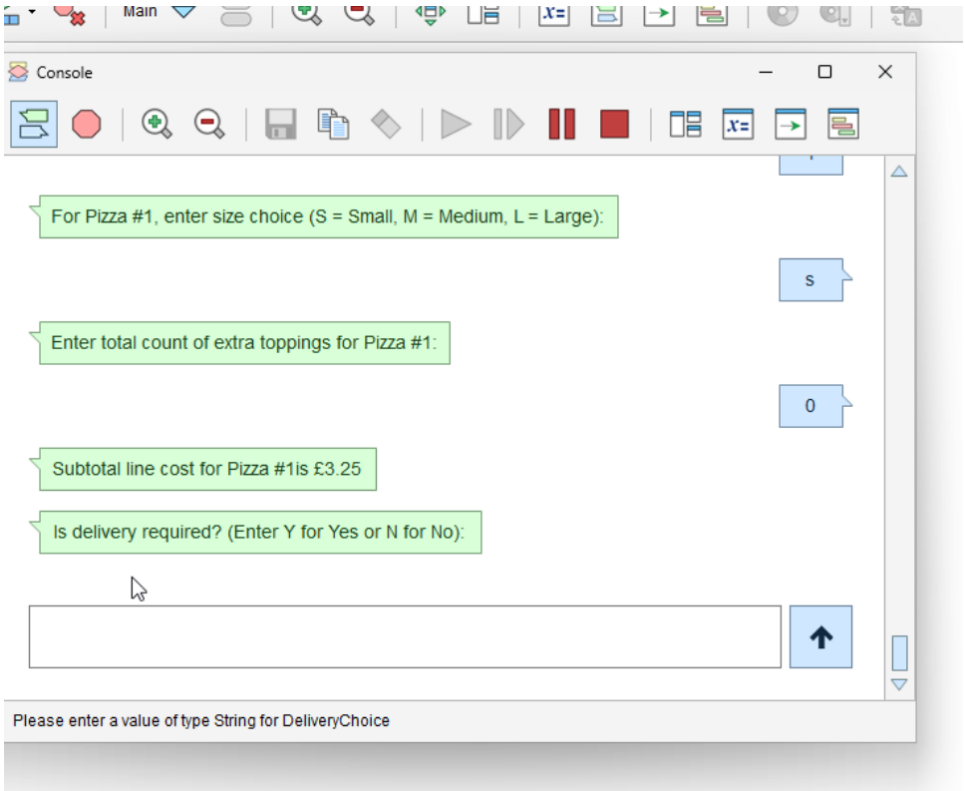


Figure 3

The program was tested using minimum valid quantity of 1 pizza. Although the input was accepted the program wrongly asked for pizza 2. this displayed price for a small pizza was also £3 instead of £3.25

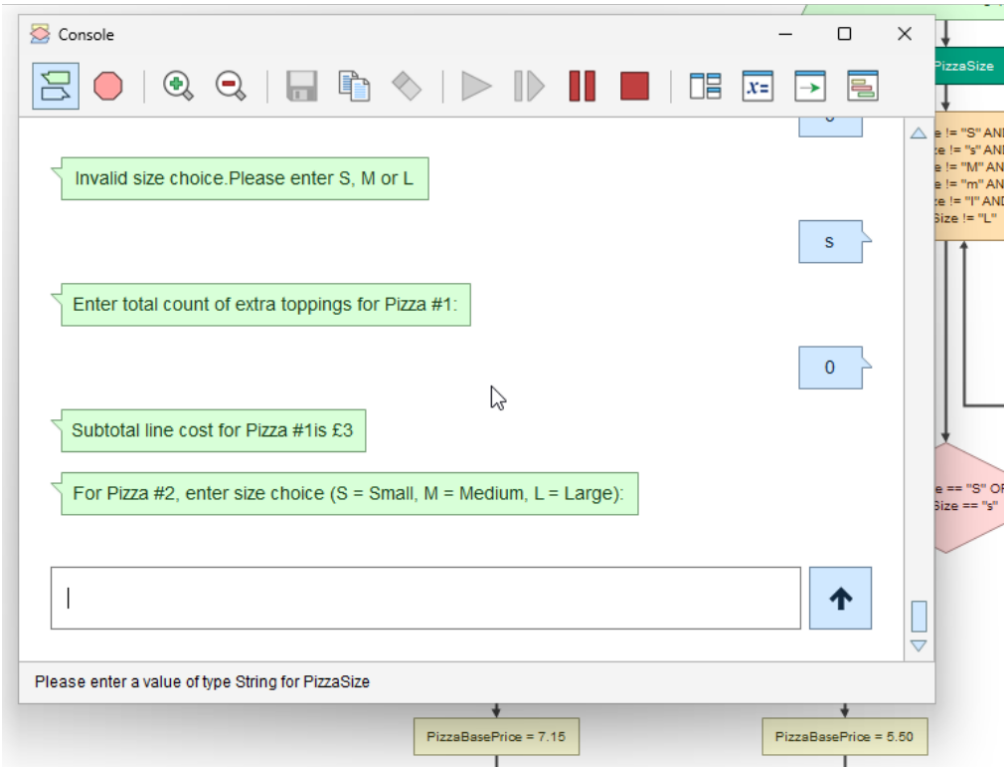


Figure 4

The program was tested using the maximum valid quantity of 6 pizzas it accepted and requested for delivery.

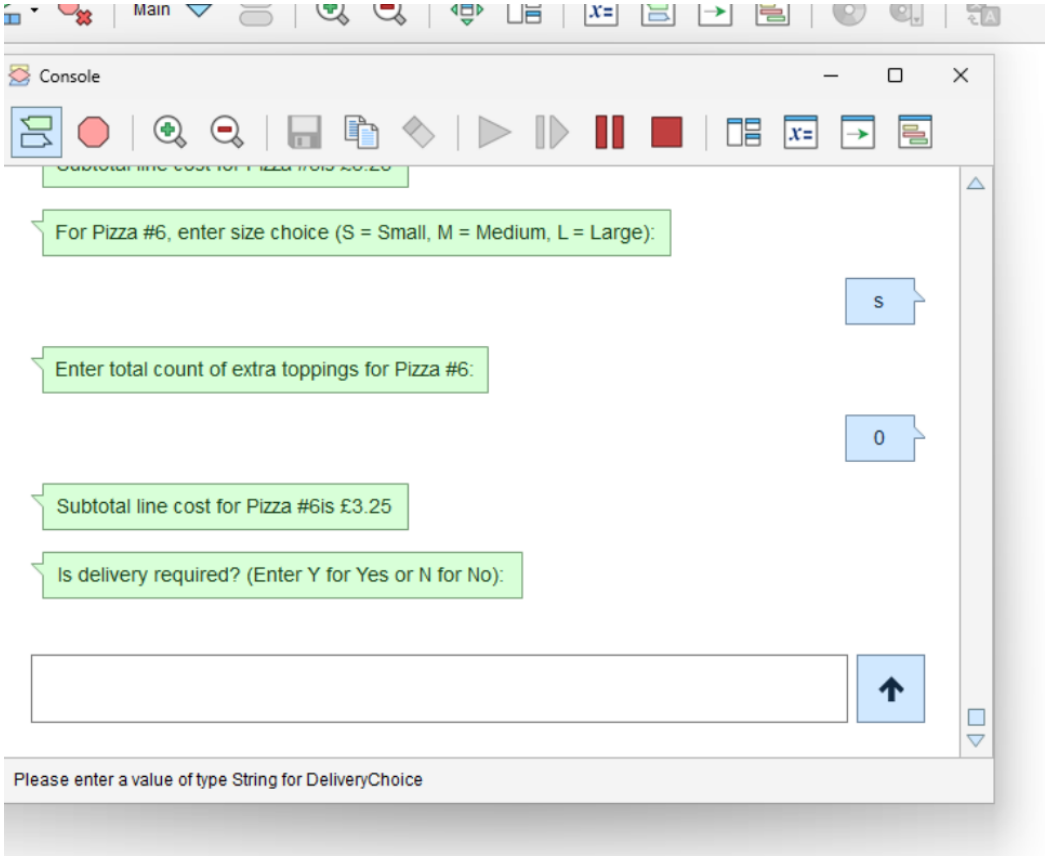

Main
Console
For Pizza #6, enter size choice (S = Small, M = Medium, L = Large):
S
Enter total count of extra toppings for Pizza #6:
0
Subtotal line cost for Pizza #6is £3.25
Is delivery required? (Enter Y for Yes or N for No):
Please enter a value of type String for DeliveryChoice

Figure 5

The program correctly rejected 7 pizzas because the maximum allowed is 6

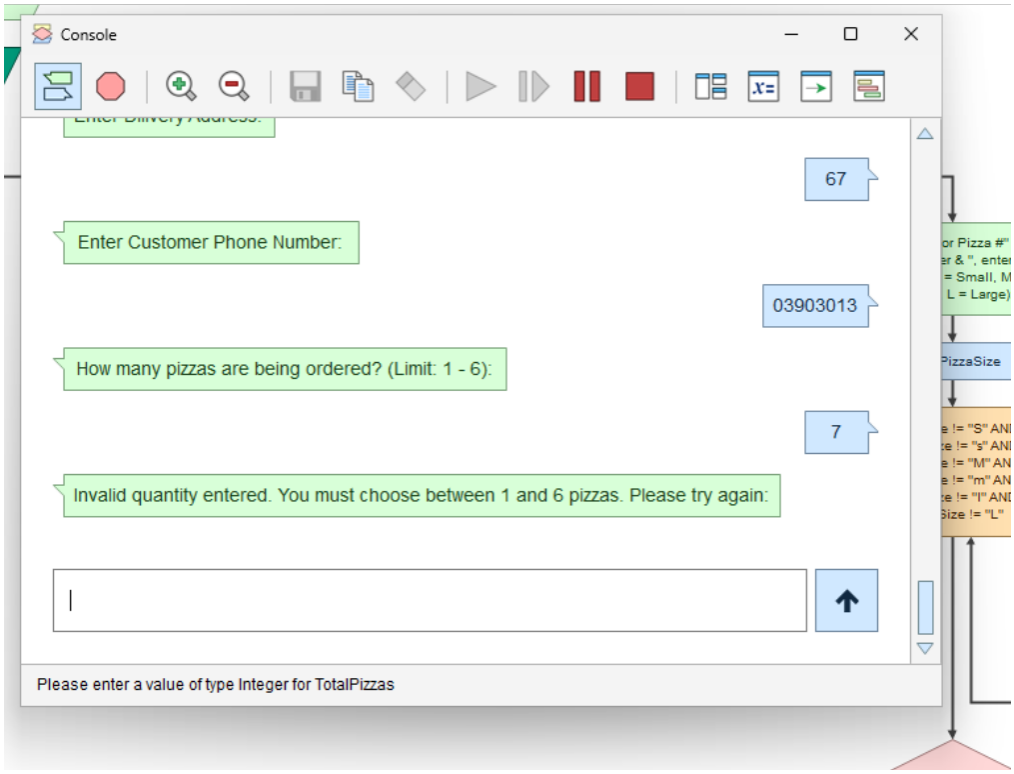


Figure 6

It asked for the correct amount pizza base price £3.25

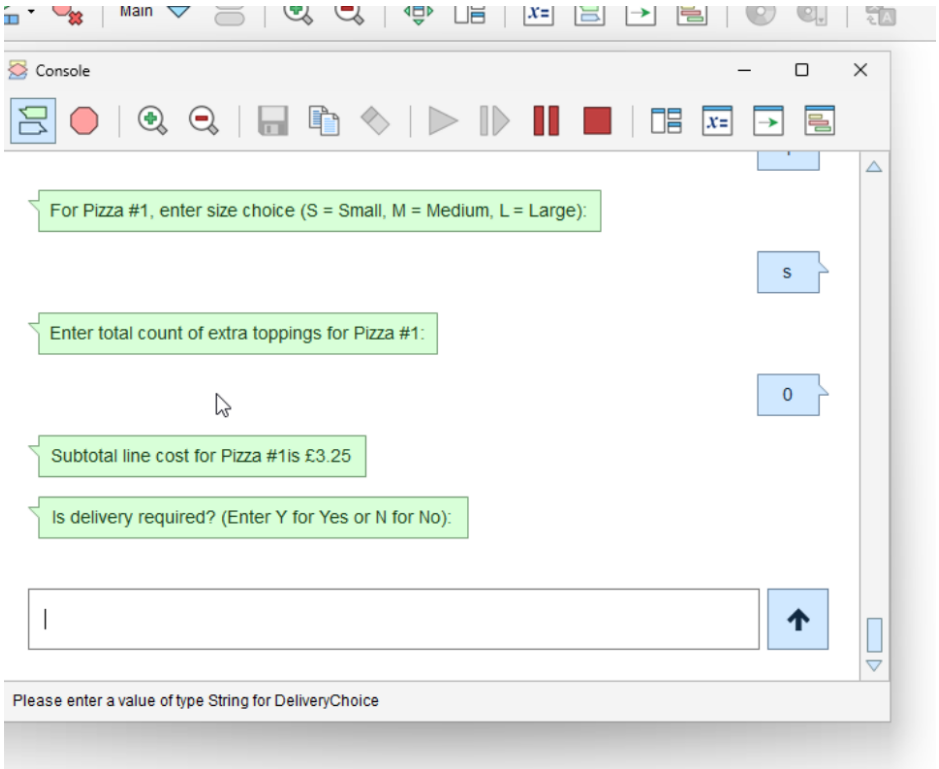


Figure 7

It asked for the correct price £5.50

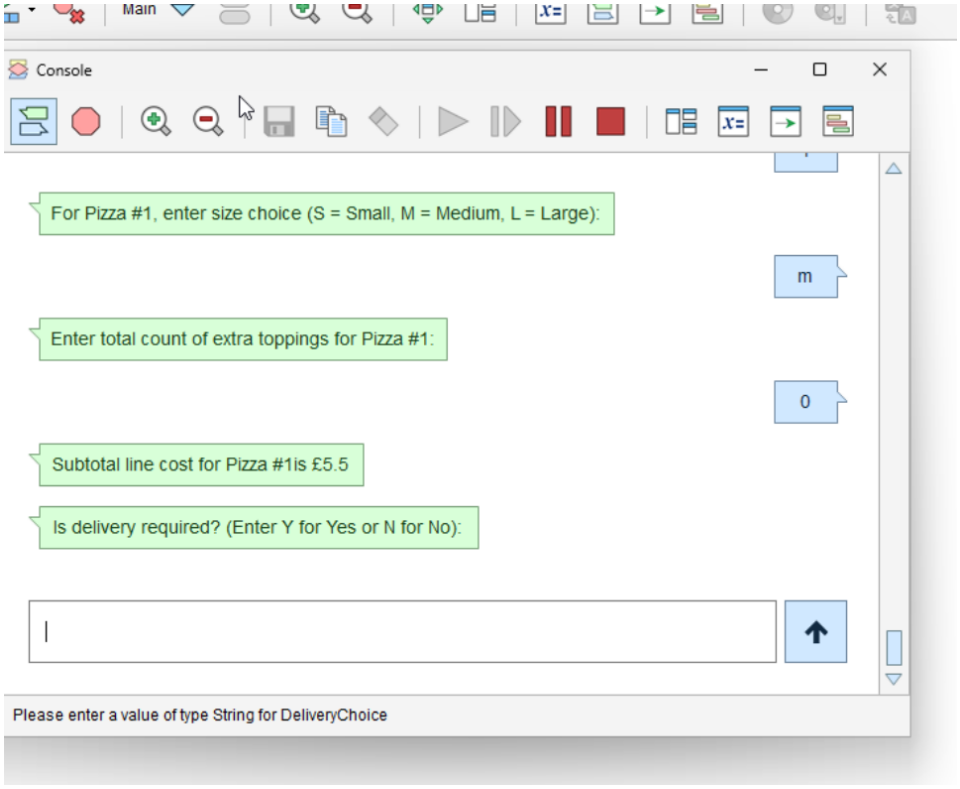


Figure 8

Program displayed correct amount £7.15

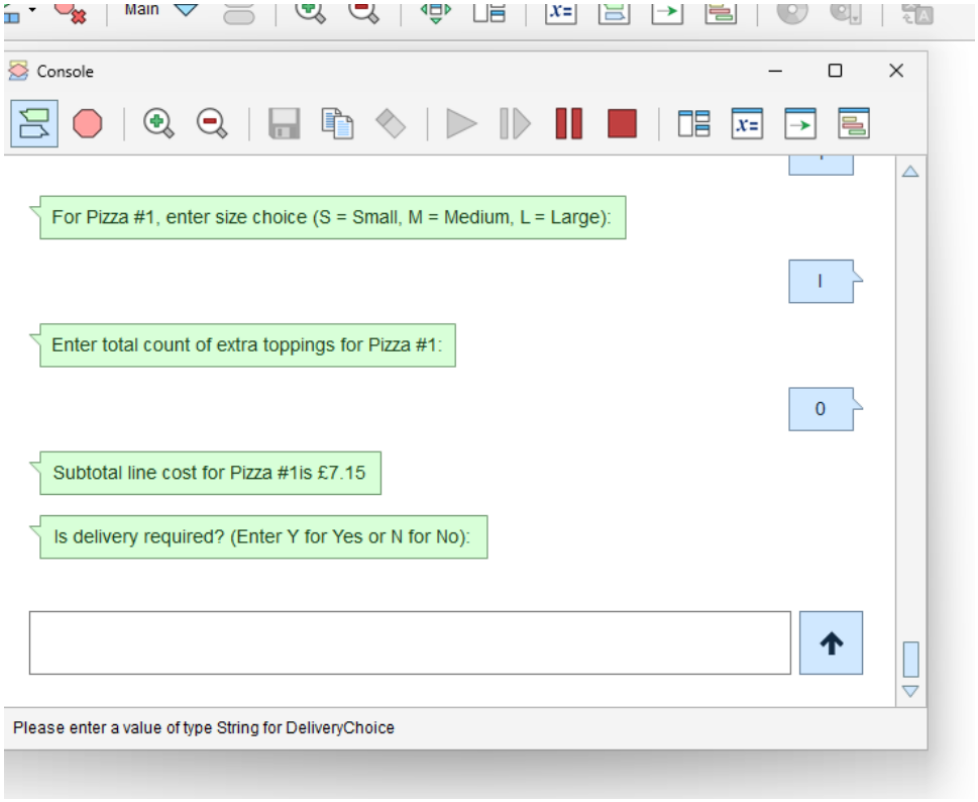


Figure 9

Program displayed the correct amount £4.60

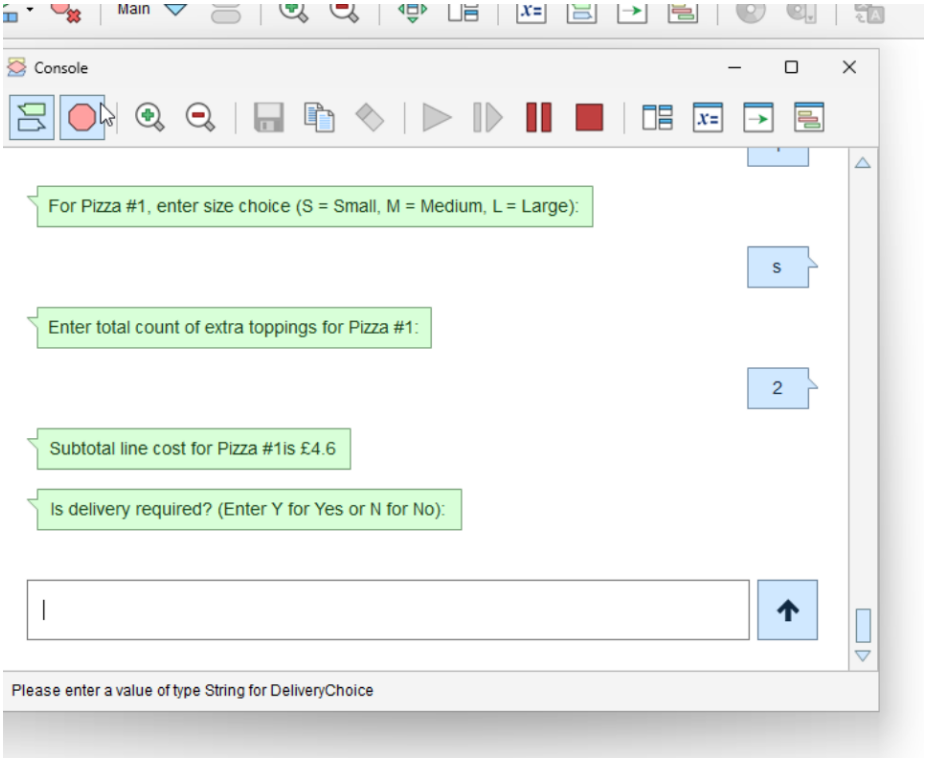


Figure 10

The program accepted 0 extra toppings and correctly calculated the small pizza cost as £3.25 without any extra charges

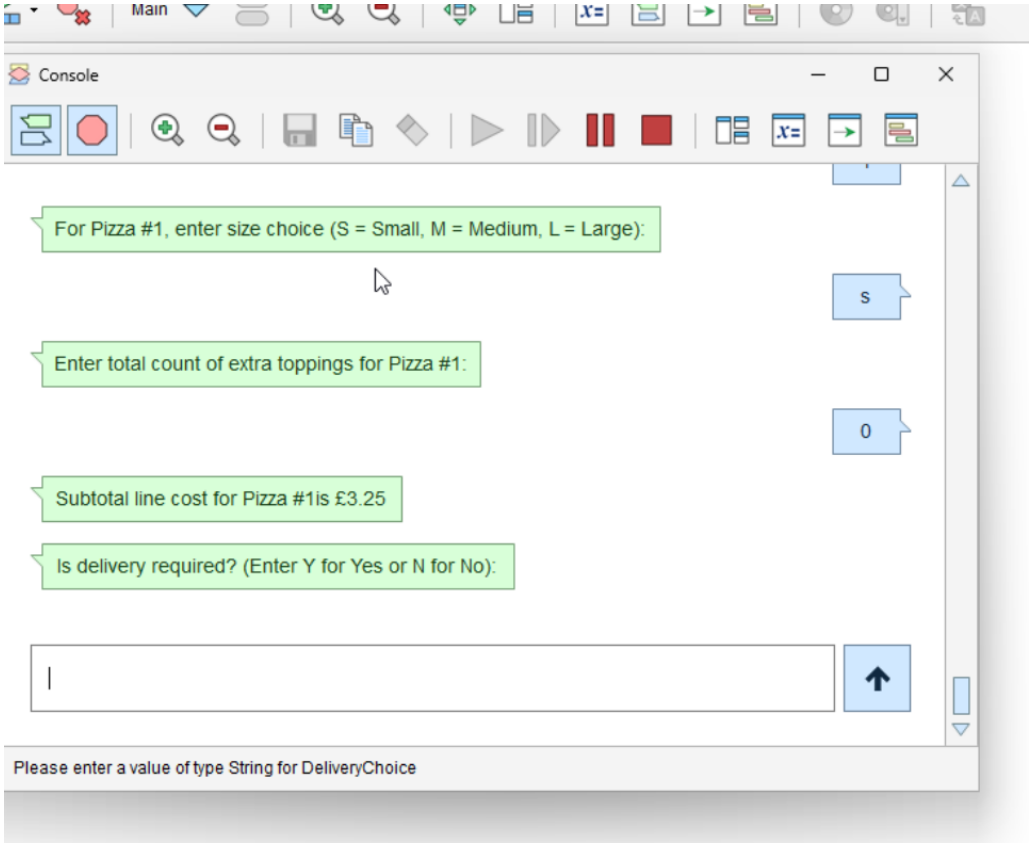


Figure 11

The program correctly added £2.50 delivery charge when the customer selected delivery.

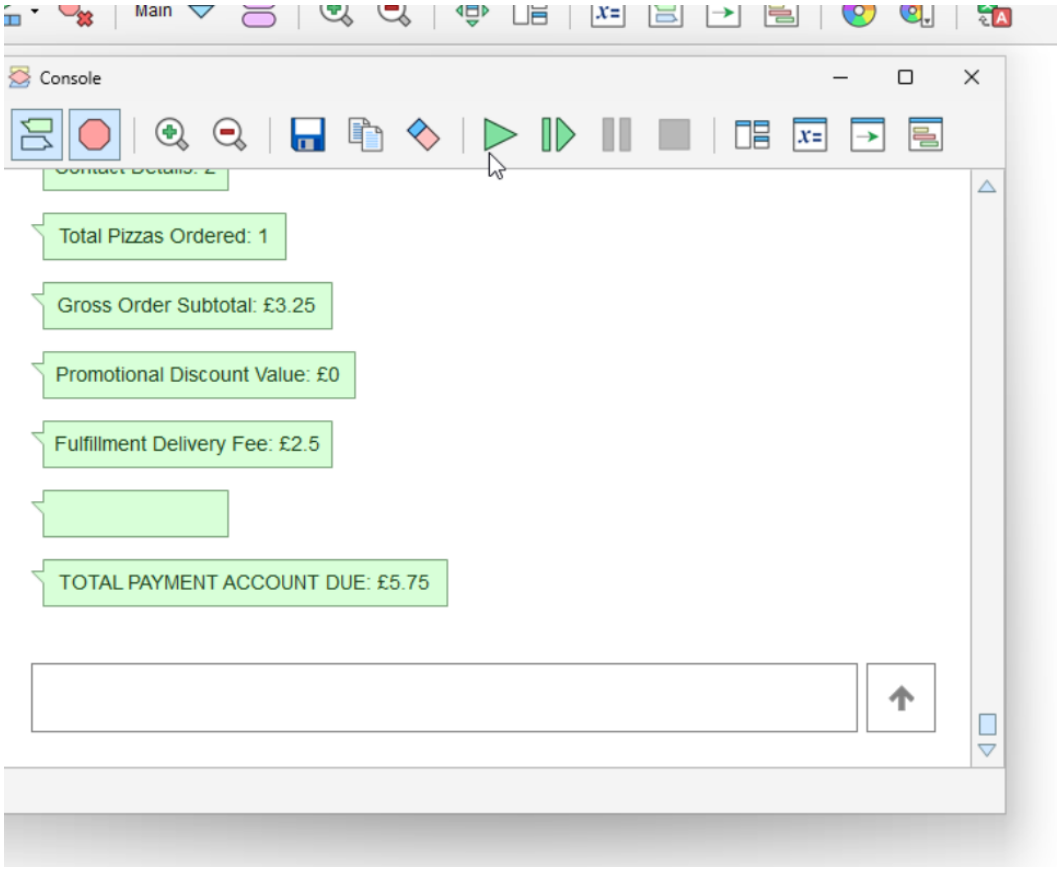


Figure 12

The program correctly calculated 10% discount of£2,74 on a subtotal of £27.40, resulting in a final total of £24.66

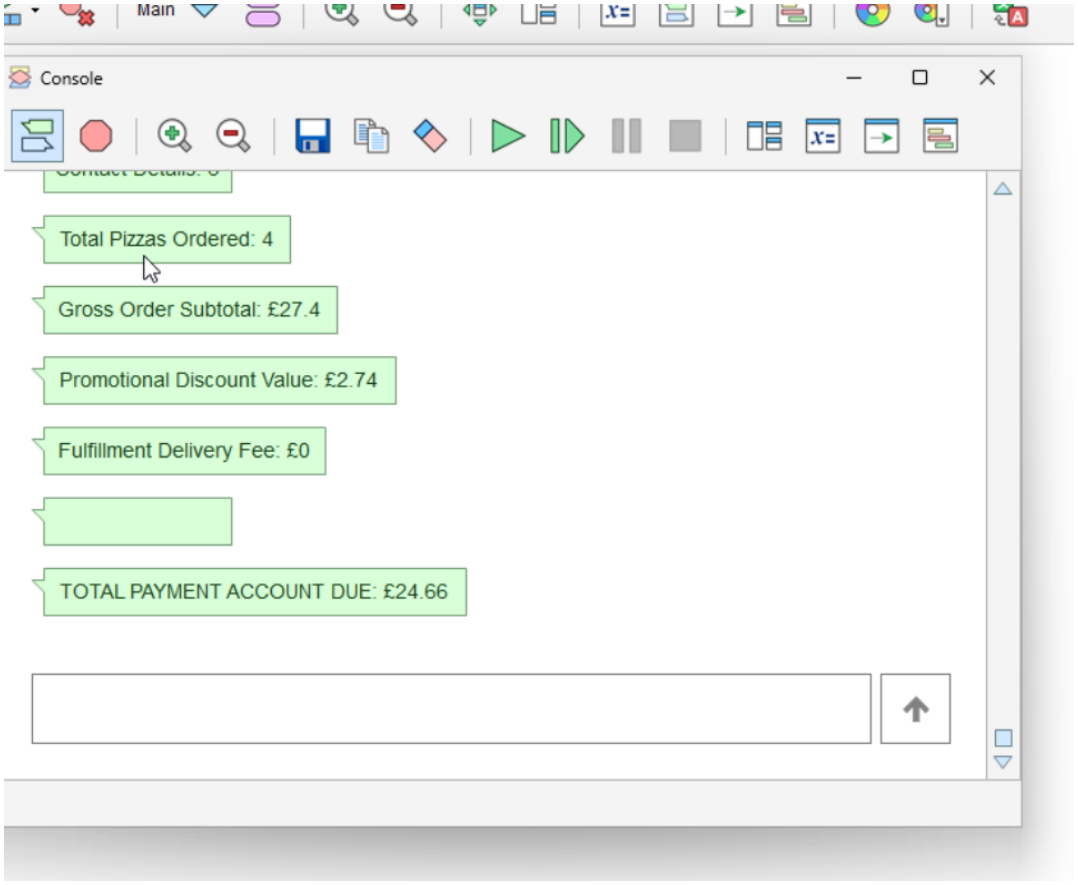


Figure 13

The program accepted a small pizza with zero extra toppings and displayed the pizza costs.

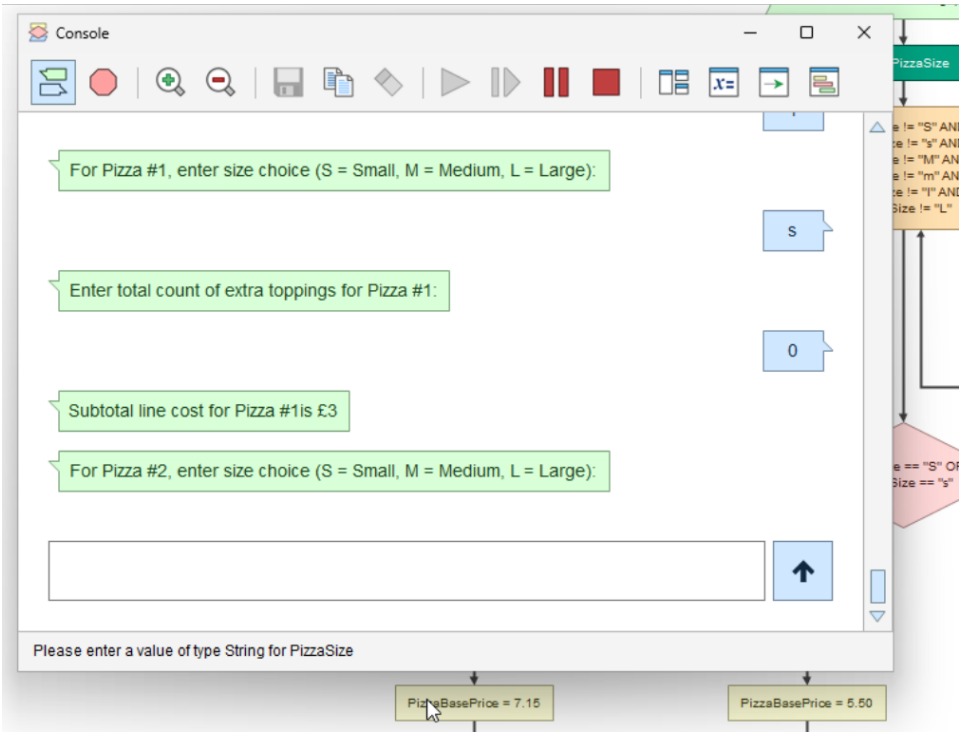


Figure 14

The program correctly applied no discount when the order subtotal was £19.75 which is below the £20 threshold.

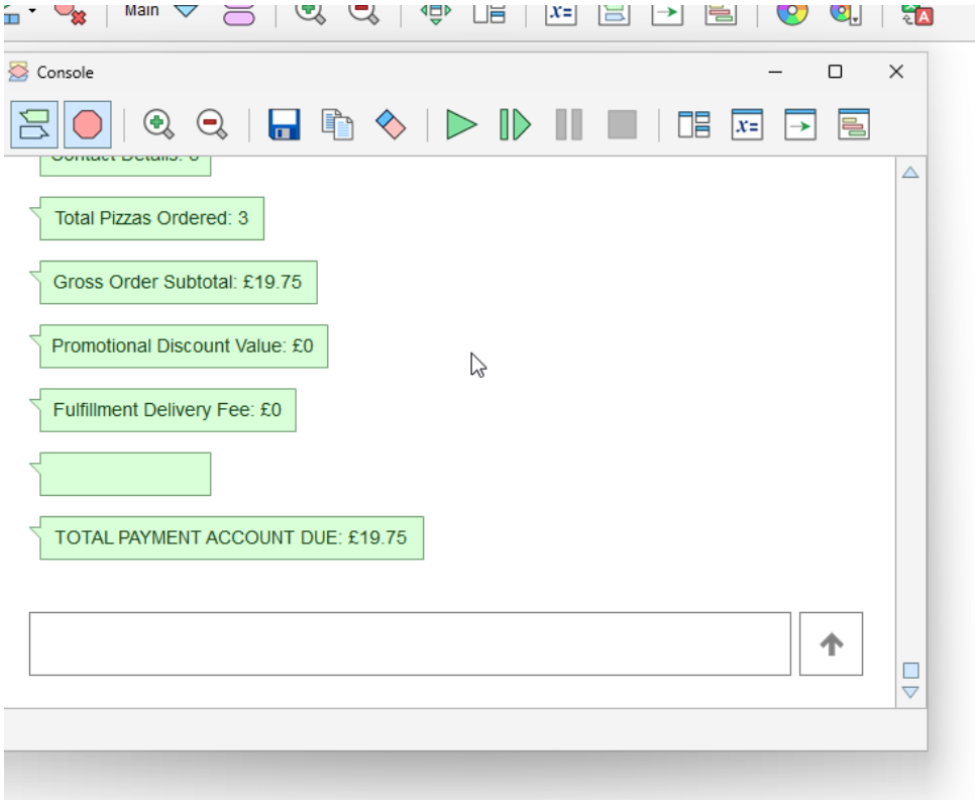
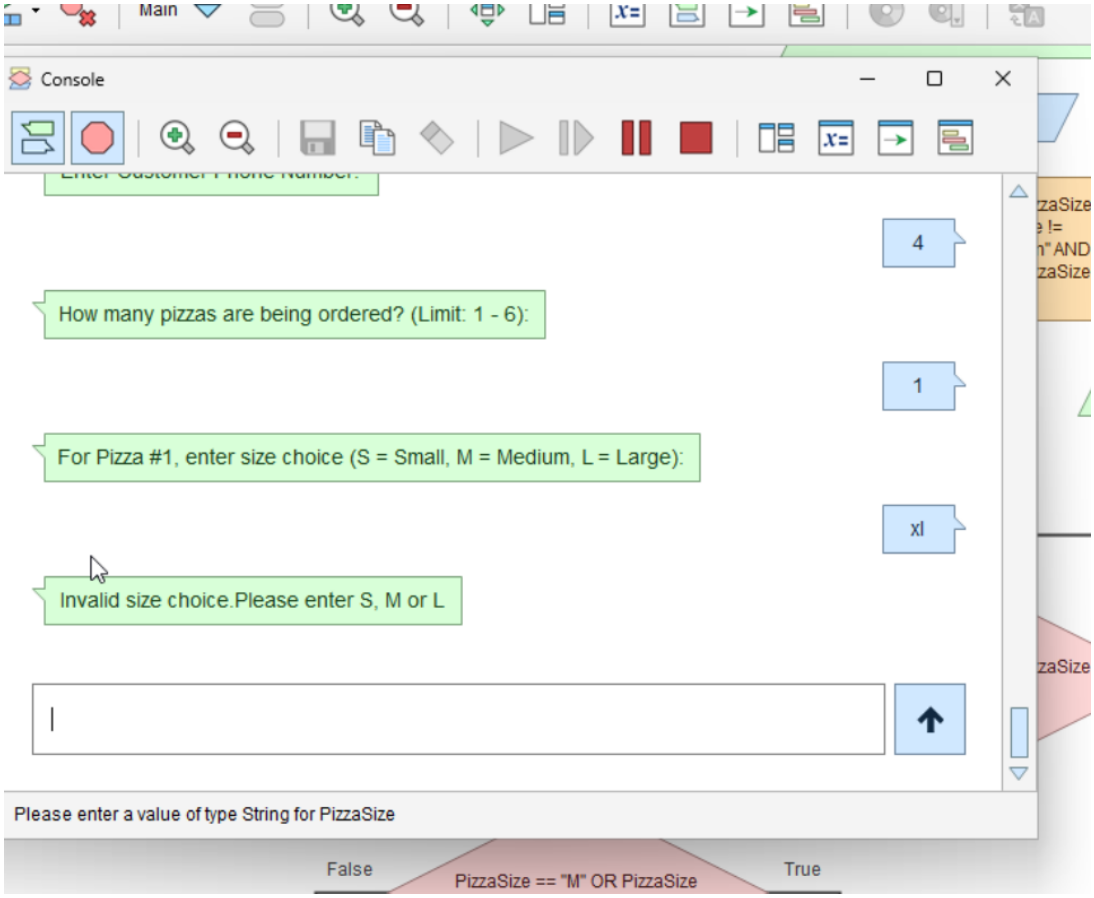


Figure 15

Program correctly rejected the invalid pizza size xl and requested a valid size of S, Mor L



Deploy:

Deployment and Execution Instructions:

1. Download and install the latest version of the Flowgorithm application.
2. Download and open the final project named pennine_pizza_works.fprg in Flowgorithm.
3. Click the green Play/Run button on the top main bar.
4. Enter the details and type in your order when prompted on the screen.

Individual Reflection - Jimmy:

My role in the team was Design. I was responsible for the subroutine list, the data plan and the planned main program flow. These parts show how the program should work before built; the rest of team could use them as a plan. The Subroutine design phase outlined seven distinct Subroutines and functions including GetValidNumber, GetSizePrice and DisplayBill. For each one I wrote what it takes in, what it gives back and which client requirements it covers from R1 to R24. I recommended using arrays in the data plan because customers can order up to 6 pizzas. Without arrays, we would need 12 individual variables to store the size and topping prices for each pizza. However, by using arrays and a FOR loop, each price can be stored efficiently. I've also checked my tables against the requirements table and correct them. For example, by adding the R24 (Validation rule).

In my data plan, I determined what information the program needs to store and chose the most suitable data type for each item. I suggested using four arrays, including a customer array that stores three String values. It holds the name, address and phone number. I used string for the phone number so that the first 0 is not lost.

This task highlighted the critical value of detail. And it helped me to understand the importance of planning before you write code. I learned how to choose the right data types and how arrays make a program shorter and easier to manage. It has also strengthened my ability to solve problems, Pay close attention to details and communicate effectively. I learned that numerical inputs must be validated, and I learned the difference between subroutines and functions; a function gives back a value like GetSizePrice While the subroutine does a job, like DisplayBill.

One challenge was that the design and the Flowgorithm were made at different times, so they did not fully match. This reinforced the vital role of communication in a group

dynamic, because each role relies on the others. Moving forward, I will share my design early so it can align with the team structure before we start building.

Individual Reflection - Faris:

My assigned role was to software test. I created 15 test cases using normal abnormal and extreme data to check the pizza ordering system. I also had to make sure my tests covered the main client requirements including topping pizza prices delivery charges and discounts.

During testing I identified errors such as the prices displaying incorrectly. I reported these issues and they were corrected in the updated program. This showed me how important it is to find the mistakes early and retest the program after changes have been done. This helped me understand software testing input validation and the high importance of checking results against the client requirements. I also improved my problem skills and teamwork. In the future I will aim to plan my tests earlier and check more possible inputs to improve my work.

Individual Reflection - Seerat:

My part in this team project was to be the Team Leader. Analysing the client requirements and assisting in the design stage by creating a requirements table, data plan and function/subroutine list. I also assisted in coordinating the team's work and make sure that tasks are done within the project schedule.

In this project I learned to appreciate more the principles of software engineering, including the need to understand the client's needs before these are met by design. The requirements table helped me to identify the inputs, processes and outputs of the system and designing the data plan helped me to understand variables, arrays and data types. It was also beneficial to me to create the list of functions and subroutines so I could see how the bigger problem can be divided into smaller and more manageable parts.

A challenge that I encountered was making sure that all the things that the client brief requested was there and that it was all properly connected to the system design. To

overcome this, I read through the business rules over and over again and used the business rules materials to help my team members ensure that the design would allow for all the necessary calculations such as pizza pricing, topping charges, discounts and delivery fees. This helped me to develop my attention to detail and planning skills.

I also discovered that it was vital to communicate, organize and cooperate as a team as I was the head of my team. Assisted in organizing contributions from team members and tracked the progress of the work. This helped me become more confident in the leadership of a group task and managing responsibilities effectively. For future projects, I will continue to develop my project planning skills and my communication skills, to accomplish work in a more efficient and higher quality manner.

Individual Reflection - Aiman:

My assigned role in this team project was the System Builder. I was fully responsible for taking the initial requirements, design plans, and business rules created by my teammates and turning them into a working computer program inside Flowgorithm.

During this project, I developed a strong practical understanding of software loops, validation pathways, and strict variable data types. The biggest challenge I faced during the build phase was a bug where the program dropped decimal values, turning a small pizza price of £3.25 into a flat £3. This error occurred because financial variables were accidentally configured as Integers instead of Reals. I successfully resolved the fault by changing the data types to Real, which restored decimal point precision to the final customer invoice. This experience taught me how critical accurate data typing is when translating real world business pricing rules into working software code.

Team Contribution Record:

Task	Faris	Jimmy	Seerat	Aiman
Requirements analysis			Responsible	
IPO chart (Input – Process – Output)	Responsible	Responsible		Responsible
Design (Data plan, function/subroutine list)		Responsible	Responsible	
Flowgorithm build (variables, input, processing, decisions, loops, output)				Responsible
Testing / error handling	Responsible			

Deployment notes				Responsible